

# ARM CORE 固件模块复用使用指南 V1.2

## 1 背景

ARM CORE STM32H723 方案核心板是加速器内部各个模块单元的数据处理、控制中心，采用核心板+底版的扩展性设计，便于软硬件功能复用。故，本文档着重介绍下嵌入式软件/固件功能复用情况。

## 2 硬件资源

嵌入式固件开发基于 ARM CORE 的单板资源，又分为片上和板载资源。

- ◆ 片上资源：STM32H723 芯片支持的功能
- ◆ 板载资源：核心板上的硬件逻辑单元，不包含 STM32H723

片上资源功能由 STM32H723 芯片提供。加速器各个模块间可复用的（无需修改，可直接使用）芯片资源如表 1，此处仅列出项目中可能用到的硬件资源。

表 1 STM32H723 片上资源

| 序号 | 名称       | 功能说明    | 是否实现 |
|----|----------|---------|------|
| 1  | FPU      | 浮点计算    | —    |
| 2  | MPU      | 内存保护    | ×    |
| 3  | CRC      | CRC 校验  | √    |
| 4  | RTC      | 实时时钟    | √    |
| 5  | FMC      | 存储器控制   | √    |
| 6  | IWDG     | 独立看门狗   | √    |
| 7  | USART1   | Console | √    |
| 8  | ETH      | 以太网     | ×    |
| 9  | WWDG     | 窗口看门狗   | √    |
| 10 | HSEM     | 硬件信号量   | √    |
| 11 | RNG      | 随机数生成   | ×    |
| 12 | USB      | USB     | ×    |
| 13 | JTAG/SWD | 下载、调试   | —    |
| 14 | BKP_RAM  | 备用 RAM  | √    |

注：

- ：无需开发
- √ ：已实现
- ×

ARM CORE 单板上除 STM32H723 外的所有逻辑芯片以及功能固定的硬件接口，均称为板载资源。

表 2 ARM CORE 板载资源

| 序号 | 名称      | 功能说明     | 占用资源类型   | 是否实现 |
|----|---------|----------|----------|------|
| 1  | LAN9252 | Ethercat | OCTOSPI1 | √    |
| 2  | W5500   | Ethernet | SPI1     | √    |
| 3  | W9825   | SDRAM    | FMC      | √    |
| 4  | LED     | 状态指示灯    | GPIO     | √    |
| 5  | FRAM    | 铁电 RAM   | SPI6     | √    |

### 3 固件功能复用

当前固件包含多个不涉及外部资源参与的功能实现，即仅需要 MCU、SRAM、FLASH 基本片上资源，如表 3 所示。

表 3 固件功能

| 序号 | 名称           | 功能说明          | 是否实现 |
|----|--------------|---------------|------|
| 1  | bootloader   | 引导程序          | √    |
| 2  | encryption   | 加密            | √    |
| 3  | ymodem       | 数据传输协议        | √    |
| 4  | backtrace    | Hard fault 回溯 | √    |
| 5  | Init export  | 初始化函数导出       | √    |
| 6  | console      | 控制台           | √    |
| 7  | gpio         | GPIO 控制接口     | √    |
| 8  | gpio imitate | GPIO 模拟协议     | √    |
| 9  | shell        | 控制台函数导出       | √    |
| 10 | ulog         | 日志            | √    |

### 4 应用层接口及使用说明

- (1) 工程架构，按各个功能单元进行分类，放在在不同的文件夹下。可通过工程根目录下的 README.md 文件查看。
- (2) 可选择性编译，进行功能裁减：注释掉 cmakefile.txt 文件中的相应路径即可。
- (3) 不同功能单元文件夹下放置了源文件和头文件，头文件中定义了外部可使用的函数。

接下来主要针对各个功能单元的对外接口函数进行说明。

## 4.1 Ethercat

### 4.1.1 设备驱动

工程中使用 LAN9252 Ethercat 协议栈，设计上（工程架构：工程修改说明.pdf）编写接口层代码（lan9252\_port.c 文件），实现硬件接口读写等操作；然后将其对接到原厂提供的 SPIDriver.c 文件相应操作函数内。

此处，通过共修改和实现如下表几个文件。

表 4 lan9252 实现文件

| 序号 | 名称                   | 来源          | 功能说明                                     |
|----|----------------------|-------------|------------------------------------------|
| 1  | 9252_HW.c            | 原厂提供        | 协议栈内部使用的读写接口，中断、定时器控制等                   |
| 2  | 9252_HW.h            | 原厂提供        | —                                        |
| 3  | lan9252_app.c        | SSC Tool 生成 | 协议栈应用层接口函数，并编写应用注册函数                     |
| 4  | lan9252_app.h        | SSC Tool 生成 | —                                        |
| 5  | lan9252_appObjects.h | SSC Tool 生成 | 数据对象                                     |
| 6  | lan9252_port.c       | 编写          | 设备定义及初始化，OCTOSPI 操作函数实现，供 SPIDriver.c 调用 |
| 7  | lan9252_port.h       | 编写          | —                                        |
| 8  | SPIDriver.c          | 原厂提供        | 驱动接口，供 9252_HW.c 读写调用                    |
| 9  | SPIDriver.h          | 原厂提供        | —                                        |

### 4.1.2 模块接口

ethercat.c 文件内已实现协议栈所需的 main\_loop 和中断处理，无需再自行创建线程处理。应用层实现具体功能时，直接调用 ethercat 文件夹下提供的接口函数，函数功能说明见表 5。

表 5 ethercat 对外接口函数

| 序号 | 名称                                  | 功能说明                                     | 参数说明                                                                  |
|----|-------------------------------------|------------------------------------------|-----------------------------------------------------------------------|
| 1  | ethercat_slave_appl_cb_register     | 注册 appl 回调函数，供协议栈 APPL_Application()函数使用 | output_event: 句柄，接收到数据时发送事件的句柄<br>event_flag: 事件标志<br>fun_cb: 注册的回调函数 |
| 2  | ethercat_rcv_data_update_with_block | 将 9252 接收队列中的数据，更新到 sDOOutputs 数组        | Timeout: 超时时间                                                         |
| 3  | ethercat_send_data_update           | 将待发送数据，更新到 sDIInputs 数组                  | buf: 数据 buffer 指针                                                     |
| 4  | ethercat_rcv_data_get               | 从 sDOOutputs 数组获取数据                      | buf: 数据 buffer 指针                                                     |
| 5  | ethercat_send_data_get              | 从 sDIInputs 数组获取数据                       | buf: 数据 buffer 指针                                                     |

### 4.1.3 使用方法

在 main\_app.c 中有关于上述模块接口函数的使用，可以参考。接口函数设计上，支持两种使用方法。

方法一：调用 ethercat\_slave\_appl\_cb\_register 进行回调函数注册，然后创建线程，等待 event\_flag；一旦接收到事件，则调用 ethercat\_rcv\_data\_update\_with\_block 进行数据更新。当需要使用 ethercat 发送数据时，调用 ethercat\_send\_data\_update 函数即可。

方法二：调用 ethercat\_rcv\_data\_update\_with\_block 函数等待接收队列有效数据，然后再通过 ethercat\_send\_data\_update 等函数进行 sDOOutputs、sDIInputs 数据操作。

## 4.2 Ethernet

### 4.2.1 设备驱动

单板使用 W5500 以太网芯片，并通过 SPI1 与 MCU 通信。架构设计上，编写 w5500\_port.c 接口文件，向上对接 W5500 协议栈，向下操作硬件驱动。

这里，涉及到改动和实现的仅有 w5500\_port 接口文件。

表 6 W5500 实现文件

| 序号 | 名称           | 来源 | 功能说明                                           |
|----|--------------|----|------------------------------------------------|
| 1  | w5500_port.c | 编写 | 设备定义及初始化，SPI 操作函数实现、注册到 W5500 协议栈，供 w5500.c 使用 |

|   |                |      |                |
|---|----------------|------|----------------|
| 2 | w5500_port.h   | 编写   | —              |
| 3 | socket.c       | 原厂提供 | socket 封装配置及读写 |
| 4 | socket.h       | 原厂提供 | —              |
| 5 | w5500.c        | 原厂提供 | 协议栈读写接口        |
| 6 | w5500.h        | 原厂提供 | —              |
| 7 | wizchip_conf.c | 原厂提供 | W5500 芯片配置函数   |
| 8 | wizchip_conf.h | 原厂提供 | —              |

### 4.2.2 模块接口

tcp\_client.c 文件内已实现 W5500 硬件配置、初始化，以及中断处理。故，无需应用层再次进行配置。

该模块对外部提供的接口函数，如表 7。

表 7 ethernet 对外接口函数

| 序号 | 名称                                 | 功能说明                         | 参数说明                               |
|----|------------------------------------|------------------------------|------------------------------------|
| 1  | tcp_client_data_rcv_get_with_block | 获取 TCP 接收到的数据                | buf: 数据 buffer 指针<br>timeout: 超时时间 |
| 2  | tcp_client_data_send               | 通过 TCP 发送数据                  | buf: 数据 buffer 指针<br>len: 数据长度     |
| 3  | tcp_establish_cb_register          | TCP 处于链接状态下的，周期性操作，周期为 100ms | fun_cb: 注册的回调函数                    |
| 4  | tcp_rcv_data_callback_register     | 注册 TCP 接收到数据后的动作             | fun_cb: 注册的回调函数                    |

### 4.2.3 使用方法

接口函数使用上，首先根据实际需求调用函数 tcp\_establish\_cb\_register 和 tcp\_rcv\_data\_callback\_register 分别注册连接、接收到数据状态下的回调函数；然后使用 tcp\_client\_data\_rcv\_get\_with\_block 获取接收到的数据，使用 tcp\_client\_data\_send 进行数据发送。

## 4.3 SDRAM

SDRAM 挂在 STM32H723 的 FMC 控制器上，实现了统一编址，即可以按普通的数据总线地址进行访问。

在 SDRAM 设计上，其不仅可以缓冲存储病案数据，亦将其用作通用的 RAM。故，可在 SDRAM 中存储数据表、作为扩展的堆栈、数据搬运的临时存放等功能使用。

鉴于此，对 SDRAM 操作进行了封装，实现了其驱动接口。对外函数接口还在进一步丰富中。

## 4.4 LED

板载共 4 个 LED 指示灯，当前各个指示灯的使用情况如表 8 所示。

表 8 led 使用汇总

| 序号 | 名称   | IO 编号 | 功能说明      |
|----|------|-------|-----------|
| 1  | LED2 | PC1   | 心跳指示灯     |
| 2  | LED3 | PC2   | 暂时未用      |
| 3  | LED4 | PD7   | 硬件看门狗喂狗指示 |
| 4  | LED6 | —     | 硬件看门狗状态指示 |

## 4.5 FRAM

FRAM 是板载的一块非易失性 RAM，具有读写速度快，可单字节操作等优点。可存储单板配置等信息。

根据硬件设计及 STM32H7 总线架构，FRAM 挂载在 SPI6 总线上，SPI6 只能使用 BDMA 进行直接访问。故而，若使用 DMA 传输，需要配置搬运的 RAM 地址位于 D3 域。

工程中创建了 FRAM 设备，并进行了初始化、注册到 ulog 记录对象，作为日志记录使用。当前无对外接口，若项目的业务存在不同的使用需求，则可调用设备对象的读写函数即可。

## 4.6 bootloader

bootloader 包含两个部分，一个是 bootloader 固件，一个是 bootloader 补丁。在功能固件中，需要实现 bootloaer 补丁，即工程下的 bootloader\_patch 文件，从而在升级过程中，实现自定义的一些操作。当前 bootloader\_patch.c 文件仅仅给出了一个框架，bootloader 规划支持 4 个补丁函数入口。

## 4.7 encryption

此处，提及的信息加密采用的是散列算法，对输入数据进行散列值计算。文件

crypto\_sha256.c 对外接口实现了 SHA256 散列算法，采用的是 STM32 官方提供的算法库。这里仅作参考，各项目模块依据需求自行修改、实现。

## 4.8 ymodem

ymodem 作为一种古老且成熟的数据传输协议（一般用于串口通信），可实现下位机与上位机之间的数据交换。正是由于其成熟性和通用性，许多串口工具已经包含了该协议接口，方便了开发人员调测。

工程中移植了 ymodem 协议，且根据项目架构对其进行了修正，通过复用 Console(COM1) 实现 FLASH 数据烧录和读取。使用时，直接在 Console 窗口发送 ymodem\_start 指令，即可查看到指令用法。

该功能无对外接口，所有的功能均封装在 ymodem.c 和 console.c 源文件内部。

## 4.9 backtrace

Backtrace 作为 Cortex-M MCU 的错误追踪工具，已经被越来越多的 RTOS 集成和使用。这里不做过多介绍，可自行学习：<https://gitee.com/Armink/CmBacktrace>

## 4.10 init export、shell

init export 以及后面介绍的 shell，均使用了函数导出操作，即编译时将函数指针编译到特定的 FLASH 段。

函数导出功能舍弃了头文件间的包含关系，所有使用该功能的源文件仅需包含 init\_call.h 或 shell.h 文件即可。

Init export 用于函数自动初始化，在 init\_call.h 文件内部定义了 6 个初始化等级，等级越低，运行顺序越靠前；同一等级之间，按函数名称字符串排序决定先后顺序。

shell 用于导出 Console 命令，方便开发人员进行驱动、模块调试，以及查看系统状态等，其功能类似于 linux 下的控制台窗口。

使用时，首先包含相应的头文件，然后用宏定义处理待导出的函数即可。

## 4.11 console

Console 使用串口 1，作为基础功能实现，从而便于开发调测。Console 与 shell 命令强相关，因为 Console 设计目的就是调用 shell 命令。当串口接收到数据后，通过 Console 线程调用 shell 接口，然后在 FLASH 中找到对应的函数指针，进而执行相应的函数。

## 4.12 gpio

工程中重新封装 gpio 目的是注册回调的方式，处理 gpio 中断。如此，可将 gpio 配置、中断处理与模块功能放在同一个文件内部，避免了文件间的变量包含引用。

该功能还未完整实现，待进一步丰富。

## 4.13 gpio imitate

gpio imitate 目的是使用 gpio 模拟通信协议，如串口协议、I2C 协议或者自定义协议。该功能当前仅在 ICM 与 DMS 间通信使用。若无需求，尽量使用成熟的通信协议。

## 4.14 ulog

ulog 属于成熟组件，实现日志记录功能。工程中对其进行了修改完善，以便于支持不同类型的输出对象。

使用时，需要关注如下几个宏定义：

表 9 ulog 宏定义控制

| 序号 | 宏定义                          | 说明                   |
|----|------------------------------|----------------------|
| 1  | #define USING_ULOG_THREAD    | 使用单独的线程进行日志写入，一般无需改动 |
| 2  | #define USING_ULOG_TIMESTAMP | 是否使用时间戳              |
| 3  | #define USING_ULOG_LEVEL_TAG | 是否使用日志等级，当前代码中未做控制   |
| 4  | #define USING_ULOG_CONSOLE   | 是否输出至控制台             |
| 5  | #define USING_ULOG_FLASH     | 是否输出至 Fram           |

对外接口：

表 10 ulog 对外接口

| 序号 | 名称                       | 功能说明                 | 参数说明                                                                                                                                                                              |
|----|--------------------------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | ulog_write_func_register | 注册 ulog write 对象操作函数 | struct ulog_write_func_info 结构体：<br>int8_t (*func_init)(void): 对象初始化<br>int8_t (*func_callback)(const uint8_t *buf, uint32_t len): 对象 write 函数<br>uint8_t index: 对象编号，适用于多个目标对象区分 |



## 5 ioc 文件配置

### 5.1 中断回调

本小节，仅针对业务需要使用中断处理或者中断事件触发条件下的，使用说明。

STM32 硬件中断处理时，使用回调函数的方式实现，以减少 xxi\_t.c 对于业务的依赖，有利于将业务层集中于特定的模块下。

使用方法：在 CubeMX（版本：6.10.0）下，Project Manager->Advanced Settings 界面最右侧，有相关硬件单元中断回调的配置选项条目。

### 5.2 FreeRTOS

在 CubeMX（版本：6.10.0）FreeRTOS 配置页下，需要配置的几个选项：

表 11

| 序号 | 名称                             | 功能说明          | debug 版本 | 最终要求 |
|----|--------------------------------|---------------|----------|------|
| 1  | ENABLE_FPU                     | FPU 相关        | Enabled  | 使能   |
| 2  | USE_APPLICATION_TASK_TAG       | 线程切换时调用       | Disabled | 禁用   |
| 3  | USE_IDLE_HOOK                  | Idle 线程钩子函数   | Enabled  | 使能   |
| 4  | USE_MALLOC_FAILED_HOOK         | Malloc 失败钩子函数 | Enabled  | 使能   |
| 5  | CHECK_FOR_STACK_OVERFLOW       | 栈溢出检测         | Enabled  | 使能   |
| 6  | GENERATE_RUN_TIME_STATS        | 统计任务耗时        | Enabled  | 使能   |
| 7  | USE_TRACE_FACILITY             | 追踪工具          |          |      |
| 8  | USE_STATS_FORMATTING_FUNCTIONS | 格式化统计         |          |      |
| 9  | USE_NEWLIB_REENTRANT           | 可重入库          | Disabled | 禁用   |

# 6 其他说明

## 6.1 资源使用情况

针对 ARM Core 核心板，提供的可复用工程包含了[第二章](#)和[第三章](#)的内容，并见下图 1。

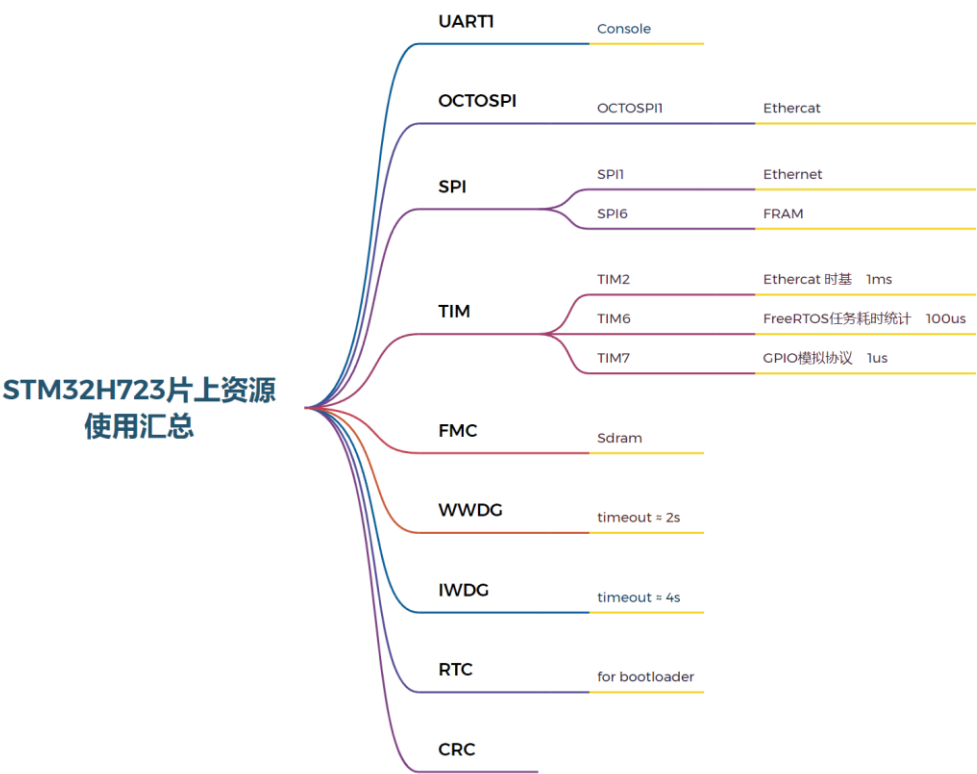


图 1 片上资源使用情况

## 6.2 Cubemx generation 后处理

针对硬件工程师不熟悉当前工程的情况，提供脚本处理文件，以实现 STM32Cubemx 重新生成工程后，直接可编译使用的目的。

使用步骤如下：

- (1) STM32Cubemx 重新生成工程文件。

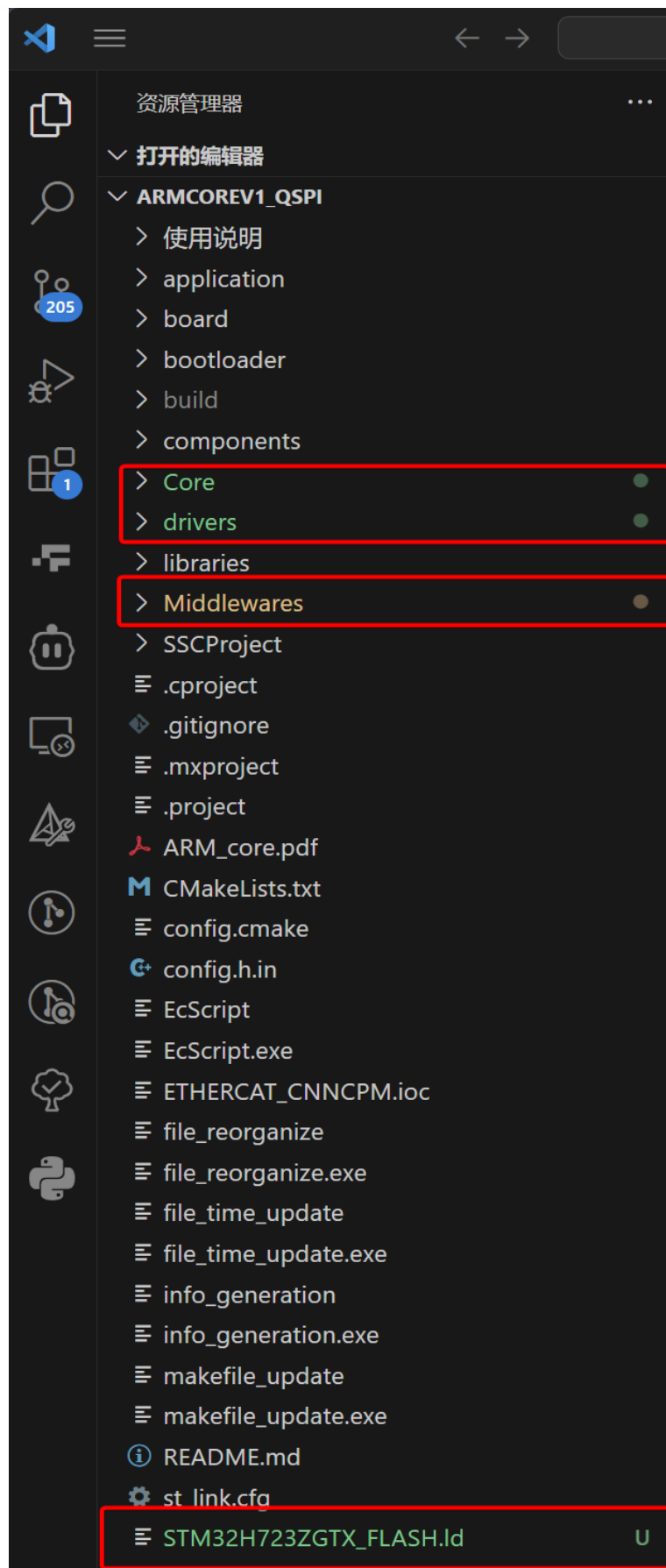


图 2 重新生成后，工程文件变更情况

(2) 使用 file\_reorganize.exe 脚本文件，重新组织文件结构。



图 3 终端窗口中，运行脚本

(3) 根据变更情况，手动修正文件。

运行脚本后，使用者需要再次修改/确认的文件如表 12。

表 12 脚本运行后修改文件

| 文件                     | 路径        | 描述          |
|------------------------|-----------|-------------|
| freertos.c             | Core/Src/ | FreeRTOS 文件 |
| main.c                 | Core/Src/ | 外设、时钟等初始化   |
| stm32h7xx_it.c         | Core/Src/ | 中断入口函数      |
| STM32H723ZGTX_FLASH.ld | /         | 链接文件        |

工程中的相应文件，应根据表 12 中的同名文件对比修改。按绝大多数业务应用需求，仅需要对比和修改 main.c 和 stm32h7xx\_it.c 两个文件即可。

(4) 修改和确认完毕后，从工程中删除表 12 中的文件。

6.3 文件结构管理及功能裁减

工程文件结构使用 cmake 进行管理，每个文件夹下存在一个 CMakeLists.txt，用以管理该文件夹下各个目录、文件的使用情况。

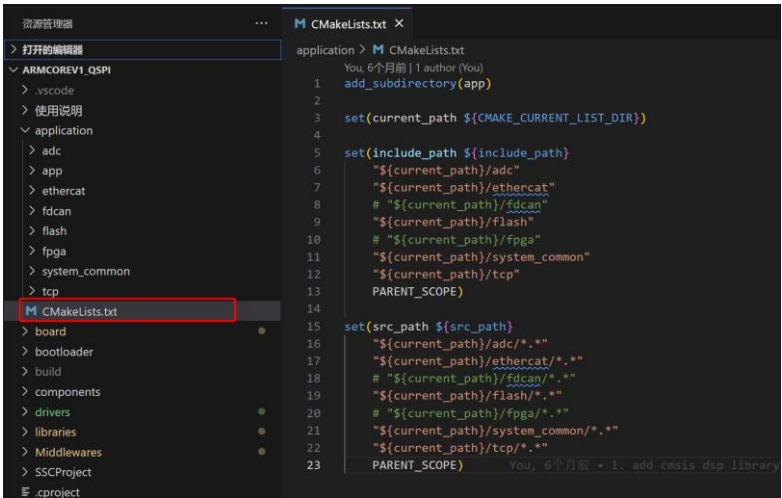


图 4 CMakeLists.txt 示例

若用户新增代码文件夹，则需要在其同级的 CMakeLists.txt 下增加相应的头文件、源码路径，如图 4；然后，使用 cmake 重新配置工程即可。

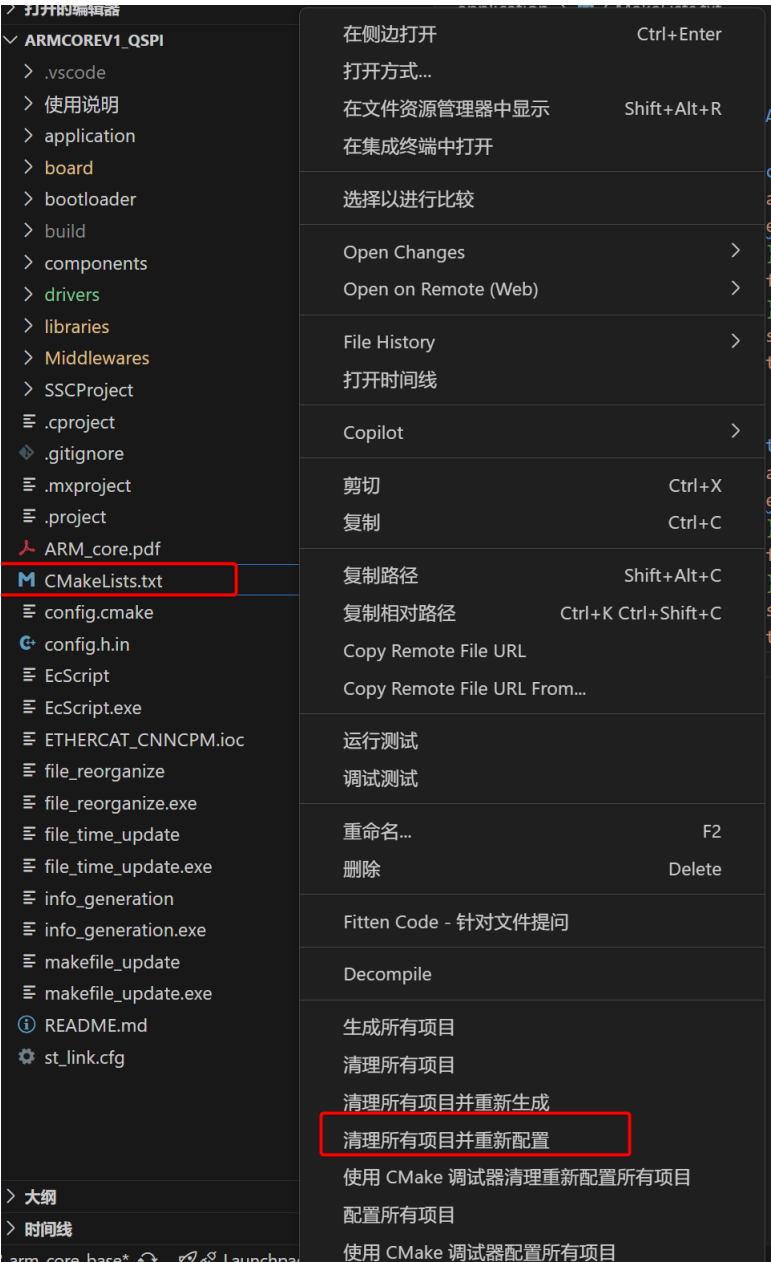


图 5 cmake 更新工程配置

同理，功能裁减时，仅需注释掉 CMakeLists.txt 中相应文件夹路径即可。

## 6.4 makefile 添加 flash download

为解决代码重新配置后，还需手动添加 make flash 指令对应的处理接口，增加了脚本文件 makefile\_update.exe，同样放置在工程的根目录下。该脚本调用已经在 cmakelists.txt 中配置，无需使用者再次操作，故务必不要删除脚本文件，且不要修改 cmakelists.txt 文件如下内容。

```

168 set(CMAKE_USER_MAKEFILE ${CMAKE_SOURCE_DIR}/makefile_update)
169
170 add_custom_command(TARGET ${PROJECT_NAME}.elf PRE_BUILD
171     COMMAND ${CMAKE_USER_MAKEFILE} -input_file_path ${MAKE_FILE} -output_file_path ${MAKE_FILE} -name ${PROJECT_NAME}
172     COMMENT "Updating ${MAKE_FILE}")
173
174 add_custom_command(TARGET ${PROJECT_NAME}.elf POST_BUILD
175     COMMAND ${CMAKE_OBJCOPY} -Oihex ${TARGET_FILE:${PROJECT_NAME}.elf} ${HEX_FILE}
176     COMMAND ${CMAKE_OBJCOPY} -Obinary ${TARGET_FILE:${PROJECT_NAME}.elf} ${BIN_FILE}
177     COMMAND ${CMAKE_USER} -o ${INFO_BIN_FILE} -k ${BIN_FILE} -n mlc_arm_code
178     COMMENT "Building ${HEX_FILE}"
179     Building ${BIN_FILE}
180     Building ${INFO_BIN_FILE}")
181

```

图 6 cmakelists 相应脚本调用配置

## 6.5 脚本文件

表 13 脚本文件说明

| 脚本文件             | 功能                      |
|------------------|-------------------------|
| EcScript         | Lan9252 PDO 数据对象对齐      |
| file_reorganize  | STM32Cubemx 工程生成后处理     |
| file_time_update | 更新固件编译时间                |
| info_generation  | 增加固件文件头，供 bootloader 使用 |
| makefile_update  | 增加 makefile 下载接口配置      |